

0.102.1. Une lettre qui tire — une cible B, et trois tirs rangés dans une struct

Le 0.102 (la cible B), avec trois tirs comme au 0.101.1. Mais les cinq tableaux d'un tir (tx, ty, tdx, tdy, vole) sont rangés ensemble : un type Tir, et un seul tableau Tir tirs[3].

1. Ce qu'il faut comprendre

C'est le 0.102, avec plusieurs tirs. Garde A enfoncé : une **rafale** part vers le B, jusqu'à **trois** tirs en vol en même temps, comme au 0.101.1.

Le problème : au 0.101.1, un tir était éparpillé dans **cinq tableaux** : tx[i], ty[i], tdx[i], tdy[i], vole[i]. Ça marche, mais rien ne dit que ces cinq cases parlent du même tir. On fait comme pour les ennemis du 0.85 : on range ensemble ce qui va ensemble.

Ce qui est nouveau ici (1) : le type Tir. struct Tir { ... }; (le 0.80) range les cinq parties d'un tir sous un seul nom : x, y, dx, dy, vole. Puis Tir tirs[3]; : un tableau de trois tirs, numérotés 0, 1 et 2. tirs[1].x est la colonne du 2e tir ; tirs[2].vole dit si le 3e vole. Au départ, tout vaut 0 : aucun ne vole.

Ce qui vient d'avant : chercher un tir libre avec lance (le 0.96.1), la rafale avec chaque(250) (le 0.97.1), la cible touchée et placerB() (le 0.102). Seule l'écriture change : tirs[i].vole au lieu de vole[i], tirs[i].x au lieu de tx[i]...

Et un tir qui touche le B ? Le cas « touché » du 0.102 est dans la boucle : vole redevient 0 pour ce tir-là, tirs[i].vole = 0, et sa place sert au tir suivant.

Avancer : le même code qu'au 0.102, dans une boucle for, avec tirs[i].x au lieu de tx, tirs[i].vole au lieu de vole... et le tir se dessine avec texte(tirs[i].x, tirs[i].y, ".").

Un tir qui en rattrape un autre ? Ils vont à la même vitesse : ça n'arrive pas. Mais si un tir trouve une case pas vide, même un autre tir, il s'y écrase : c'est le cas 3, rien à ajouter.

Essaie : passe à cinq tirs : Tir tirs[5];, et i < 5 dans les deux boucles. Et chaque(150) pour une rafale plus serrée.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <hasard>` → [0.81.2. La fonction hasard\(\) — un nombre au hasard](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <texte>` → [0.0.2. La fonction texte\(\) — écrire un mot](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <note>` → [0.82.2. La fonction note\(\) — jouer une note](#)
- `#include <reste>` → [0.8.4. #include <reste> — a % b](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2197 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	105 o	← la console, à l'allumage - la mise en route de la console
chaque_minuteur()	124 o	← main() - une fonction fournie par le langage
placerB()	55 o	← main() - ta fonction
main()	1182 o	← démarrage - ta fonction
VBlank	8 o	← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage	28 o	← image() - attend l'image suivante
AttendreVBlank	16 o	← démarrage - attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireTexte, EcrireNombre - attend de pouvoir écrire une case
AdresseCase	10 o	← lire(), poser(), effacer(), texte() - trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() - efface des cases
EcrireTexte	10 o	← texte() - écrit un texte, lettre par lettre
EcrireNombre	33 o	← nombre() - écrit un nombre en chiffres
Reste	10 o	← placerB() - le reste d'une division
CopierTuiles	85 o	← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage - vide les deux cartes de l'écran
Hasard	25 o	← hasard() - tire un nombre au hasard
LireManette	37 o	← bouton() - lit les huit boutons
Tuiles	304 o	← CopierTuiles - 19 lettres sur 43
Notes	120 o	← note() - la table des notes de musique
"SCORE"	5 o	← texte() - un texte gravé
"."	1 o	← texte() - un texte gravé
"."	1 o	← texte() - un texte gravé

Lettres gravées : ABCEORSX0123456789.

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, Diviser, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, EffacerFond, EffacerPanneau, DonneesTransfert

Fonctions du langage appelées :

texte()	x 3	147 o sur place, dans main() + AdresseCase (10 o), EcrireTexte (10 o)
effacer()	x 3	95 o sur place, dans main() + AdresseCase (10 o), EffacerCases (12 o)
poser()	x 5	64 o sur place, dans placerB(), main() + AdresseCase (10 o)
note()	x 1	56 o sur place, dans main() + Notes (120 o)
lire()	x 4	44 o sur place, dans placerB(), main() + AdresseCase (10 o)
bouton()	x 5	25 o sur place, dans main() + LireManette (37 o)
nombre()	x 2	20 o sur place, dans main() + EcrireNombre (33 o)
hasard()	x 2	6 o sur place, dans placerB() + Hasard (25 o)
image()	x 1	3 o sur place, dans main() + AttendreImage (28 o)
images()	x 1	2 o sur place, dans chaque_minuteur()

Tes fonctions appelées :

```
chaque_minuteur() x 3    32 o sur place, dans main()
placerB()          x 1    3 o sur place, dans main()
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UNE LETTRE QUI TIRE : trois tirs à la fois ----
// Le 0.102, et le A peut avoir TROIS tirs en vol en même temps.
// Chaque tir a sa place, son sens, et son « vole » : on les range
// dans un tableau de struct (le 0.85).

// ---- NOUVEAU : le type Tir, tout ce qu'il faut savoir d'UN tir ----

#include <hasard>      // tire un nombre au hasard
#include <lire>        // lit la tuile posée sur une case
#include <poser>       // pose une tuile sur une case du fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <texte>       // écrit un texte à l'écran
#include <nombre>      // écrit un nombre en chiffres
#include <chaque>      // répond 1 toutes les n millisecondes
#include <bouton>      // lit un bouton de la manette
#include <effacer>     // efface des cases, ou tout le fond
#include <note>        // joue une note
#include <reste>       // a % b, sauf par 1, 2, 4, 8, 16... écrits en clair
```

2. UN ENREGISTREMENT — TIR

```
struct Tir {
    uint8_t x, y;           // sa place
    uint8_t dx, dy;        // son sens
    uint8_t vole;          // 1 = en vol ; 0 = libre, prêt à partir
};

// ---- NOUVEAU : trois tirs, numérotés 0, 1 et 2 ----
// tirs[0].x : la colonne du 1er tir ; tirs[2].vole : le 3e vole-t-il ?
// Au départ, toutes leurs parties valent 0 : aucun ne vole.
```

3. UN TABLEAU — TIRS

```
Tir tirs[3];
```

4. UNE VARIABLE — AX

```
uint8_t ax = 2;           // le A : sa colonne...
```

5. UNE VARIABLE — AY

```
uint8_t ay = 8;           // ...et sa ligne
```

6. UNE VARIABLE — DX

```
uint8_t dx = 1;           // le sens où regarde le A
```

7. UNE VARIABLE — DY

```
uint8_t dy = 0;
```

8. UNE VARIABLE — NX

```
uint8_t nx = 0;           // une case d'ARRIVÉE, calculée avant de bouger
```

9. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

10. UNE VARIABLE — BX

```
uint8_t bx = 16;           // la cible B
```

11. UNE VARIABLE — BY

```
uint8_t by = 2;
```

12. UNE VARIABLE — SCORE

```
uint8_t score = 0;
```

13. UNE VARIABLE — LANCE

```
uint8_t lance = 0;         // NOUVEAU : 1 = un tir vient de partir, à ce tour-ci
```

14. UNE FONCTION — PLACERB()

```
void placerB() {           // un B au hasard, sur une case vide (le 0.102)
    do {
        bx = hasard() % 20;
        by = hasard() % 17;
    } while (lire(bx, by) != 0);
    poser(bx, by, ALPHABET[1]);
}
```

15. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    for (uint8_t l = 4; l < 13; l++) {
        poser(12, l, ALPHABET[23]);
    }
    poser(ax, ay, ALPHABET[0]);
    poser(bx, by, ALPHABET[1]);
    texte(0, 17, "SCORE");
    nombre(6, 17, score);

    while (true) {
        image();

        // ---- le A bouge et retient où il regarde (le 0.100) ----
        if (chaque(150)) {
            nx = ax;
            ny = ay;
            if (bouton(DROITE)) {
                nx = ax + 1;
                dx = 1;
                dy = 0;
            } else if (bouton(GAUCHE)) {
                nx = ax - 1;
                dx = -1;
                dy = 0;
            } else if (bouton(BAS)) {
                ny = ay + 1;
                dx = 0;
                dy = 1;
            } else if (bouton(HAUT)) {
                ny = ay - 1;
                dx = 0;
                dy = -1;
            }
            if (nx < 20 && ny < 17) {
                if (lire(nx, ny) == 0) {
                    effacer(ax, ay, 1);
                    ax = nx;
                    ay = ny;
                    poser(ax, ay, ALPHABET[0]);
                }
            }
        }

        // ---- NOUVEAU : A lance un tir, au plus toutes les 250 ms ----
        // Sans ce rythme, garder A enfoncé lancerait les trois tirs en trois
        // images, collés les uns aux autres. Avec chaque(250) : une rafale.
        if (chaque(250)) {
            if (bouton(A)) {
                nx = ax + dx;
                ny = ay + dy;
                if (nx < 20 && ny < 17 && lire(nx, ny) == 0) {
                    // On cherche un tir LIBRE (vole == 0), et on lance le premier trouvé.
                    // lance passe à 1 dès qu'un tir est parti : les suivants, même
                    // libres, ne partent pas à ce tour-ci. Un seul tir par appui.
                    lance = 0;
                    for (uint8_t i = 0; i < 3; i++) {
                        if (lance == 0 && tirs[i].vole == 0) {
                            tirs[i].vole = 1;
                            tirs[i].x = nx;
                            tirs[i].y = ny;
                            tirs[i].dx = dx;
                            tirs[i].dy = dy;
                            texte(tirs[i].x, tirs[i].y, ".");
                            lance = 1;
                        }
                    }
                }
            }
        }
    }
}
```

```

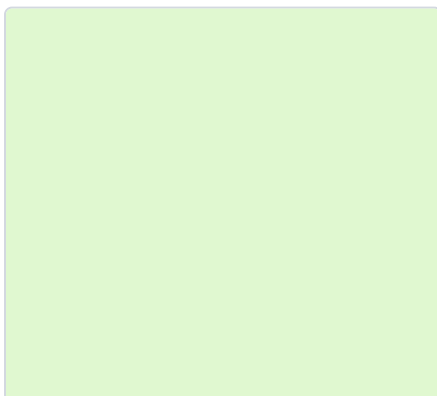
    }
    }
    // Les trois volent déjà ? lance reste à 0 : rien ne part.
}
}

// ---- NOUVEAU : CHAQUE tir avance : le 0.102, dans une boucle ----
// Le même code qu'avant, avec tirs[i].x au lieu de tx, tirs[i].vole
// au lieu de vole... i vaut 0, puis 1, puis 2.
if (chaque(100)) {
    for (uint8_t i = 0; i < 3; i++) {
        if (tirs[i].vole == 1) {
            nx = tirs[i].x + tirs[i].dx;
            ny = tirs[i].y + tirs[i].dy;
            effacer(tirs[i].x, tirs[i].y, 1);
            if (nx >= 20 || ny >= 17) {
                tirs[i].vole = 0;          // 1. hors de l'écran
            } else if (nx == bx && ny == by) {
                tirs[i].vole = 0;          // 2. la cible : touché !
                effacer(bx, by, 1);
                score = score + 1;
                nombre(6, 17, score);
                note(1, D05, 10, 12);
                placerB();
            } else if (lire(nx, ny) != 0) {
                tirs[i].vole = 0;          // 3. un X, ou un autre tir : il s'écrase
            } else {
                tirs[i].x = nx;             // 4. une case vide : il y va
                tirs[i].y = ny;
                texte(tirs[i].x, tirs[i].y, ".");
            }
        }
    }
}
}
}
}
}

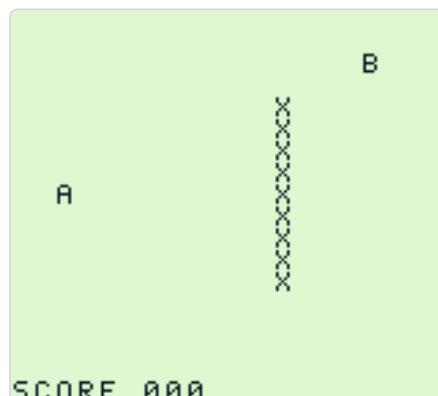
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



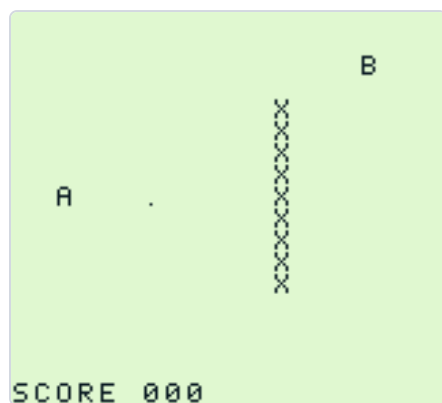
la toute première image — la console vient de démarrer — image 1



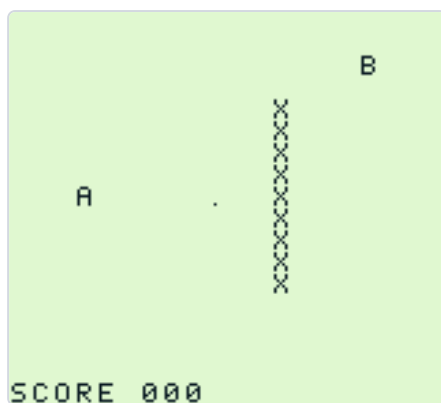
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

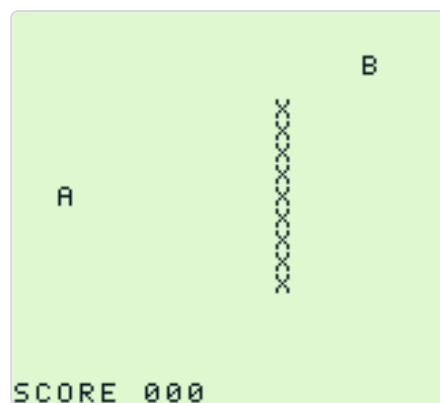
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



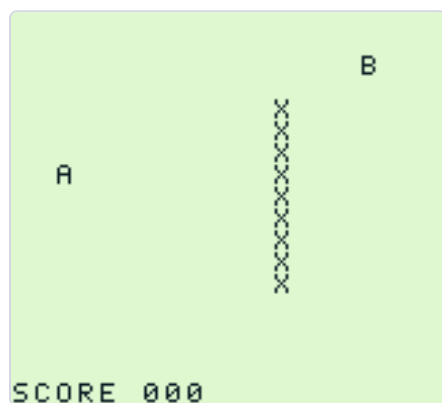
A — l'écran a changé



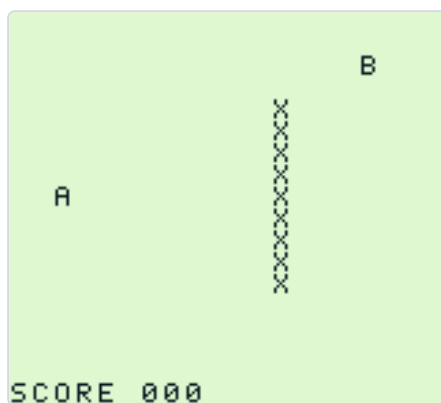
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

A enfoncé : une rafale de « . », espacés, jusqu'à trois en vol en même temps ; chacun s'écrase sur le mur ou touche le B.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ A tenu : trois tirs en vol en même temps, pas plus (au plus 3)
- ✓ les tirs partent espacés, jamais collés
- ✓ ils s'écrasent tous sur le mur
- ✓ la rafale touche le B : le score monte (score 1)

Ensuite : **0.103. Une lettre qui tire — le B tire aussi** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++